TCP/IP Socket Fundamentals - Building Network Applications

Theory, Edge Cases & Code Examples

- ✓ What is Network Programming?
- ✓ The Foundation
- ✓ Your Network Toolkit
- ✓ Server vs Client Architecture
- ✓ Finding Your Way on the Network

→ Swipe to learn →

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 What is Network Programming?

Network programming is the art of making different computers talk to each other over a network. Just like how you make a phone call to talk to someone far away, network programming allows programs running on different machines to communicate and share data. In C++, we use **sockets** as the fundamental building block for network communication.

Think of a socket as a telephone: you pick it up, dial a number (connect), talk (send/receive data), and hang up (close). The socket API provides these exact operations for programs to communicate over networks.

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 Understanding TCP/IP - The Foundation

TCP/IP is actually two protocols working together:

IP (Internet Protocol) is like the postal service. It knows how to find the destination address and route your data packet there. IP doesn't guarantee delivery—it's like dropping a postcard in a mailbox. It might arrive, it might not, and if you send multiple postcards, they might arrive out of order.

TCP (Transmission Control Protocol) sits on top of IP and adds reliability. TCP is like sending a registered letter with tracking and confirmation. It ensures:
- Your data arrives intact (error checking)
- Data arrives in the correct order (sequencing)
- The receiver acknowledges receipt (acknowledgments)
- Lost packets are automatically resent (retransmission)

For autonomous vehicles communicating with cloud services, TCP/IP ensures critical data like HD maps, traffic updates, and trajectory planning commands arrive reliably and in order.

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 The Socket API - Your Network Toolkit

A socket is an endpoint for sending or receiving data across a network. Every networked application uses sockets. When you browse the web, your browser creates a socket to talk to the web server. When you stream video, a socket carries that data. Understanding sockets is understanding how all networked software works.

The socket API provides these key operations:

socket() - Creates a new socket (like buying a new phone)
bind() - Assigns an address to the socket (like getting a phone number)
listen() - Prepares to accept connections (like turning on your phone's ringer)
accept() - Accepts an incoming connection (like answering the phone)
connect() - Initiates a connection to a server (like making a call)
send()/recv() - Sends and receives data (like talking)
close() - Closes the connection (like hanging up)

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 Server vs Client Architecture

Network applications typically use a client-server model:

Server: Listens for incoming connections, waits for clients to connect, serves requests. Servers run continuously, waiting for work. Think of a server as a restaurant kitchen—always open, waiting for orders.

Client: Initiates connections, sends requests, receives responses. Clients are active when needed. Think of a client as a customer—comes in, orders food, eats, leaves.

For autonomous vehicles:
- **Vehicle (Client)**: Connects to cloud services, requests HD map updates, sends telemetry
- **Cloud Server (Server)**: Waits for vehicle connections, provides map data, processes route calculations

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 Address Structures - Finding Your Way on the Network

Every device on a network needs an address. In TCP/IP, we use IP addresses and port numbers together to identify a specific application on a specific machine.

sockaddr_in structure contains:
- **sin_family**: Address family (AF_INET for IPv4)
- **sin_port**: Port number (16-bit, identifies the application)
- **sin_addr**: IP address (32-bit, identifies the machine)

IP Address: Identifies the computer (like a street address)
Port Number: Identifies the specific application (like an apartment number)

Example: `192.168.1.100:8080`
- `192.168.1.100` = which computer
- `8080` = which application on that computer

Common ports:
- 80: HTTP (web servers)
- 443: HTTPS (secure web)
- 22: SSH (secure shell)
- 5000-9000: Custom applications (like your autonomous vehicle service)

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 Network Byte Order - Speaking the Same Language

Different computer architectures store multi-byte numbers differently:
- **Big-endian**: Stores most significant byte first (network standard)
- **Little-endian**: Stores least significant byte first (Intel x86/x64)

Example: The number 0x1234 in memory:
- Big-endian: [0x12] [0x34]
- Little-endian: [0x34] [0x12]

Network protocols mandate big-endian (network byte order). Use these conversion functions:
- **htons()**: Host to network short (16-bit port numbers)
- **htonl()**: Host to network long (32-bit IP addresses)
- **ntohs()**: Network to host short
- **ntohl()**: Network to host long

Always convert to network byte order before sending, convert back after receiving. Forgetting this causes subtle bugs where connections fail only between different architectures.

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 Blocking vs Non-Blocking Sockets

Blocking sockets (default behavior):
When you call recv() or send(), your program **stops and waits** until the operation completes. It's like waiting at a red light—you can't do anything else until it turns green.

```
char buffer[1024];
int bytes = recv(socket_fd, buffer, sizeof(buffer), 0);  // ⏸️ Blocks here until data arrives
```

Non-blocking sockets:
Operations return immediately, even if they can't complete. You get an error code like EAGAIN (try again later). It's like checking if a parking spot is free—if not, you move on immediately rather than waiting.

Blocking sockets are simpler for single-connection scenarios. Non-blocking sockets are essential for high-performance servers handling thousands of connections (we'll explore this with select/poll/epoll).

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 Socket Lifecycle - The Complete Journey

Server lifecycle:

```
socket() → Create a communication endpoint
    ↓
bind() → Assign an address (IP + port)
    ↓
listen() → Mark as passive socket (ready to accept connections)
    ↓
accept() → Block until client connects, returns new socket for communication
    ↓
recv()/send() → Exchange data with client
    ↓
close() → Clean up resources
```

Client lifecycle:

```
socket() → Create a communication endpoint
    ↓
connect() → Initiate connection to server
    ↓
send()/recv() → Exchange data with server
    ↓
close() → Clean up resources
```

TCP/IP Socket Fundamentals - Building Network Applications

Theory

📌 Why Socket Programming Matters for Systems Engineers

Understanding sockets is fundamental because:

1. **Distributed Systems**: Microservices communicate via network protocols
2. **Real-Time Communication**: WebSockets, game servers, chat applications
3. **IoT and Embedded**: Devices communicating with cloud services
4. **Performance Tuning**: Network I/O is often the bottleneck—understanding sockets helps optimize
5. **Debugging**: Network issues are common in production—socket knowledge is essential for troubleshooting

In autonomous driving:
- Vehicles stream sensor data (lidar, camera) to edge compute servers
- Real-time map updates from cloud to vehicle
- Vehicle-to-vehicle (V2V) communication for cooperative perception
- OTA (over-the-air) software updates
- Fleet management and telemetry

All of these use sockets under the hood.

TCP/IP Socket Fundamentals - Building Network Applications

Edge Cases

📌 1. SIGPIPE - The Silent Killer

```
int socket_fd = /* connected socket */;
char message[] = "Hello";

// Client closes connection
// Server writes to closed socket
int bytes = send(socket_fd, message, 5, 0);  // ❌ SIGPIPE signal kills process!
```

When you write to a socket whose peer has closed the connection, the kernel sends a **SIGPIPE signal** to your process. The default action is **process termination**, which crashes your server!

Why this happens: TCP has already sent a RST (reset) packet from the closed peer. When you try to send again, the kernel knows the connection is dead and sends SIGPIPE to notify your program.

Solution 1: Ignore SIGPIPE and check return values

```
#include <signal.h>

int main() {
    signal(SIGPIPE, SIG_IGN);  // ✅ Ignore SIGPIPE

    // send() now returns -1 with errno = EPIPE instead of crashing
    int bytes = send(socket_fd, message, 5, 0);
    if (bytes < 0 && errno == EPIPE) {
        std::cerr << "Client disconnected\n";
        close(socket_fd);
    }
}
```

Solution 2: Use MSG_NOSIGNAL flag

```
int bytes = send(socket_fd, message, 5, MSG_NOSIGNAL);  // ✅ Never generates SIGPIPE
if (bytes < 0 && errno == EPIPE) {
    // Handle disconnection
}
```

For production servers, always use MSG_NOSIGNAL or ignore SIGPIPE globally. Otherwise, a single client disconnect can crash your entire server.

TCP/IP Socket Fundamentals - Building Network Applications

Edge Cases

📌 2. Partial Sends and Receives (1/2)

```
char buffer[10000];
memset(buffer, 'A', sizeof(buffer));

int bytes_sent = send(socket_fd, buffer, 10000, 0);
std::cout << "Sent: " << bytes_sent << " bytes\n";  // ❌ Might be only 6144!
```

send() and recv() do not guarantee to transfer all requested bytes in one call. They return the number of bytes actually transferred, which might be less than requested due to:
- Network buffer space limitations
- Kernel buffer size
- Network congestion
- Receiver's processing speed

Correct pattern for sending all data:

```
ssize_t send_all(int fd, const char* buffer, size_t length) {
    size_t total_sent = 0;
    while (total_sent < length) {
        ssize_t bytes = send(fd, buffer + total_sent, length - total_sent, MSG_NOSIGNAL);
        if (bytes < 0) {
            if (errno == EINTR) continue;  // Interrupted by signal, retry
            return -1;  // Real error
        }
        if (bytes == 0) return total_sent;  // Connection closed
        total_sent += bytes;
    }
    return total_sent;
}
```

Similarly for receiving:

```
ssize_t recv_all(int fd, char* buffer, size_t length) {
    size_t total_received = 0;
    while (total_received < length) {
        ssize_t bytes = recv(fd, buffer + total_received, length - total_received, 0);
        if (bytes < 0) {
            if (errno == EINTR) continue;
// ...continues on next slide
```

📌 2. Partial Sends and Receives (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            return -1;
        }
        if (bytes == 0) return total_received;  // EOF - connection closed
        total_received += bytes;
    }
    return total_received;
}
```

Key insight: Always loop until all data is sent/received, or use message framing protocols.

TCP/IP Socket Fundamentals - Building Network Applications

Edge Cases

📌 3. Port Already in Use (EADDRINUSE)

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
bind(server_fd, (sockaddr*)&addr, sizeof(addr));  // ❌ bind: Address already in use
```

After a server crashes or restarts, the port enters **TIME_WAIT** state (typically 60 seconds on Linux). The kernel keeps the port reserved to handle any delayed packets from previous connections. This prevents immediate restart of your server!

Solution: Use SO_REUSEADDR socket option

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));  // ✅ Reuse port

bind(server_fd, (sockaddr*)&addr, sizeof(addr));  // Now succeeds
```

SO_REUSEADDR allows:
- Binding to a port in TIME_WAIT state
- Multiple sockets binding to the same port (with SO_REUSEPORT)
- Faster development cycle (no waiting for port release)

Critical for production: Always set SO_REUSEADDR on server sockets before binding.

TCP/IP Socket Fundamentals - Building Network Applications

Edge Cases

📌 4. Forgetting to Convert Byte Order

```
sockaddr_in server_addr{};
server_addr.sin_port = 5000;  // ❌ WRONG! Host byte order

// On little-endian (x86), 5000 = 0x1388
// Stored as: [0x88] [0x13]
// Network expects: [0x13] [0x88]
// Actual port used: 0x8813 = 34835 (wrong port!)
```

This is a sneaky bug because:
- On big-endian machines (rare nowadays), it works accidentally
- On little-endian machines (x86/x64), ports get scrambled
- Error messages say "connection refused" without mentioning byte order

Correct approach:

```
server_addr.sin_port = htons(5000);  // ✅ Convert to network byte order
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);  // ✅ For IP addresses too
```

Debugging tip: If your server listens on port 5000 but you see it on port 34835 (`netstat` output), you forgot htons().

TCP/IP Socket Fundamentals - Building Network Applications

Edge Cases

📌 5. Zombie Sockets and File Descriptor Leaks (1/2)

```
void handle_client(int client_fd) {
    if (some_error) {
        return;  // ❌ Forgot close(client_fd) - FD leak!
    }
    // ... handle client ...
    close(client_fd);  // ✅ Only reached on success path
}
```

Each process has a **limited number of file descriptors** (typically 1024 by default, check with `ulimit -n`). If you forget to close sockets:
- File descriptors are exhausted
- New connections fail with EMFILE (too many open files)
- Server stops accepting connections but doesn't crash (silent failure)

Leak accumulation example:

```
Client 1 connects: FD 3 (not closed properly)
Client 2 connects: FD 4 (not closed)
...
Client 1021 connects: FD 1023 (last available FD)
Client 1022 connects: accept() returns -1, errno = EMFILE
```

Solution: Use RAII to ensure cleanup

```
class SocketGuard {
    int fd_;
public:
    explicit SocketGuard(int fd) : fd_(fd) {}
    ~SocketGuard() { if (fd_ >= 0) close(fd_); }
    SocketGuard(const SocketGuard&) = delete;
    int get() const { return fd_; }
};

void handle_client(int client_fd) {
    SocketGuard guard(client_fd);  // ✅ Automatically closed on any return path

    if (some_error) {
// ...continues on next slide
```

📌 5. Zombie Sockets and File Descriptor Leaks (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        return;  // close() called automatically
    }
    // ... handle client ...
}  // close() called automatically
```

Monitoring tip: Watch `/proc/<pid>/fd/` to see open file descriptors. If count keeps growing, you have a leak.

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 1. Simple Echo Server (Single Client) (1/3)

This demonstrates the complete server lifecycle. The server accepts one client, echoes received messages back, and exits when the client disconnects. Notice error checking on every system call—essential for production code.

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main() {
    // Step 1: Create socket
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket failed");
        return 1;
    }

    // Step 2: Set socket options (reuse address)
    int opt = 1;
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
        perror("setsockopt failed");
        close(server_fd);
        return 1;
    }

    // Step 3: Bind to address
    sockaddr_in server_addr{};
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;  // Listen on all interfaces
    server_addr.sin_port = htons(5000);  // Port 5000

    if (bind(server_fd, (sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind failed");
        close(server_fd);
        return 1;
    }

    // Step 4: Listen for connections
    if (listen(server_fd, 5) < 0) {  // Backlog of 5
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 1. Simple Echo Server (Single Client) (2/3)

```
// ...continued from previous slide
        perror("listen failed");
        close(server_fd);
        return 1;
    }

    std::cout << "Echo server listening on port 5000...\n";

    // Step 5: Accept client connection
    sockaddr_in client_addr{};
    socklen_t client_len = sizeof(client_addr);
    int client_fd = accept(server_fd, (sockaddr*)&client_addr, &client_len);
    if (client_fd < 0) {
        perror("accept failed");
        close(server_fd);
        return 1;
    }

    char client_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, INET_ADDRSTRLEN);
    std::cout << "Client connected from " << client_ip
              << ":" << ntohs(client_addr.sin_port) << "\n";

    // Step 6: Echo loop
    char buffer[1024];
    while (true) {
        int bytes_received = recv(client_fd, buffer, sizeof(buffer) - 1, 0);
        if (bytes_received < 0) {
            perror("recv failed");
            break;
        }
        if (bytes_received == 0) {
            std::cout << "Client disconnected\n";
            break;
        }

        buffer[bytes_received] = '\0';
        std::cout << "Received: " << buffer << "\n";

        // Echo back to client
        int bytes_sent = send(client_fd, buffer, bytes_received, MSG_NOSIGNAL);
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 1. Simple Echo Server (Single Client) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        if (bytes_sent < 0) {
            perror("send failed");
            break;
        }
    }

    // Step 7: Cleanup
    close(client_fd);
    close(server_fd);
    return 0;
}
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 2. Simple TCP Client (1/2)

Client is simpler: create socket, connect, send/receive, close. The connect() call blocks until the connection is established or fails. For autonomous vehicles, the client might send GPS coordinates and receive navigation waypoints.

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main() {
    // Step 1: Create socket
    int client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd < 0) {
        perror("socket failed");
        return 1;
    }

    // Step 2: Specify server address
    sockaddr_in server_addr{};
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(5000);  // Server port

    // Convert IP address from text to binary
    if (inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr) <= 0) {
        perror("invalid address");
        close(client_fd);
        return 1;
    }

    // Step 3: Connect to server
    if (connect(client_fd, (sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("connection failed");
        close(client_fd);
        return 1;
    }

    std::cout << "Connected to server\n";

    // Step 4: Send message
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 2. Simple TCP Client (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    const char* message = "Hello from client";
    int bytes_sent = send(client_fd, message, strlen(message), MSG_NOSIGNAL);
    if (bytes_sent < 0) {
        perror("send failed");
        close(client_fd);
        return 1;
    }

    std::cout << "Sent: " << message << "\n";

    // Step 5: Receive response
    char buffer[1024];
    int bytes_received = recv(client_fd, buffer, sizeof(buffer) - 1, 0);
    if (bytes_received < 0) {
        perror("recv failed");
        close(client_fd);
        return 1;
    }

    buffer[bytes_received] = '\0';
    std::cout << "Received: " << buffer << "\n";

    // Step 6: Cleanup
    close(client_fd);
    return 0;
}
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 3. Robust send_all Function (1/2)

This handles partial sends and interrupted system calls (EINTR). Production code should always use send_all rather than raw send() for reliable data transfer.

```
#include <sys/socket.h>
#include <errno.h>

// Send exactly 'length' bytes, handling partial sends and interrupts
ssize_t send_all(int socket_fd, const void* buffer, size_t length) {
    const char* ptr = static_cast<const char*>(buffer);
    size_t total_sent = 0;

    while (total_sent < length) {
        ssize_t bytes = send(socket_fd, ptr + total_sent,
                             length - total_sent, MSG_NOSIGNAL);

        if (bytes < 0) {
            if (errno == EINTR) {
                // Interrupted by signal, retry
                continue;
            }
            // Real error (EPIPE, ECONNRESET, etc.)
            return -1;
        }

        if (bytes == 0) {
            // Connection closed by peer
            return total_sent;
        }

        total_sent += bytes;
    }

    return total_sent;  // Successfully sent all data
}

// Usage example
void send_message(int client_fd) {
    const char* message = "This is a long message that might require multiple send() calls";
    ssize_t result = send_all(client_fd, message, strlen(message));
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 3. Robust send_all Function (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    if (result < 0) {
        perror("send_all failed");
    } else if (result < strlen(message)) {
        std::cerr << "Connection closed, only sent " << result << " bytes\n";
    } else {
        std::cout << "Successfully sent all " << result << " bytes\n";
    }
}
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 4. Receiving Fixed-Length Messages (1/2)

This demonstrates message framing: sending a header with length information, then the payload. Essential for protocols where messages have variable length.

```
// Receive exactly 'length' bytes, handling partial receives
ssize_t recv_all(int socket_fd, void* buffer, size_t length) {
    char* ptr = static_cast<char*>(buffer);
    size_t total_received = 0;

    while (total_received < length) {
        ssize_t bytes = recv(socket_fd, ptr + total_received,
                             length - total_received, 0);

        if (bytes < 0) {
            if (errno == EINTR) {
                // Interrupted by signal, retry
                continue;
            }
            // Real error
            return -1;
        }

        if (bytes == 0) {
            // Connection closed (EOF)
            return total_received;  // Return what we received so far
        }

        total_received += bytes;
    }

    return total_received;
}

// Example: Receiving a structured message
struct MessageHeader {
    uint32_t message_id;
    uint32_t payload_length;
};

bool receive_message(int socket_fd) {
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 4. Receiving Fixed-Length Messages (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    MessageHeader header;

    // Receive fixed-size header
    ssize_t bytes = recv_all(socket_fd, &header, sizeof(header));
    if (bytes != sizeof(header)) {
        std::cerr << "Failed to receive complete header\n";
        return false;
    }

    // Convert from network byte order
    header.message_id = ntohl(header.message_id);
    header.payload_length = ntohl(header.payload_length);

    std::cout << "Message ID: " << header.message_id
              << ", Payload length: " << header.payload_length << "\n";

    // Receive variable-length payload
    std::vector<char> payload(header.payload_length);
    bytes = recv_all(socket_fd, payload.data(), header.payload_length);
    if (bytes != header.payload_length) {
        std::cerr << "Failed to receive complete payload\n";
        return false;
    }

    std::cout << "Received payload: "
              << std::string(payload.begin(), payload.end()) << "\n";
    return true;
}
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 5. Non-Blocking Socket with Error Handling (1/2)

Non-blocking sockets return immediately with EAGAIN/EWOULDBLOCK if no data is available. This allows your program to do other work instead of waiting. Essential for servers handling multiple connections efficiently.

```
#include <fcntl.h>
#include <sys/socket.h>
#include <errno.h>
#include <iostream>

// Make socket non-blocking
bool set_nonblocking(int socket_fd) {
    int flags = fcntl(socket_fd, F_GETFL, 0);
    if (flags < 0) {
        perror("fcntl F_GETFL");
        return false;
    }

    if (fcntl(socket_fd, F_SETFL, flags | O_NONBLOCK) < 0) {
        perror("fcntl F_SETFL");
        return false;
    }

    return true;
}

void non_blocking_example(int socket_fd) {
    set_nonblocking(socket_fd);

    char buffer[1024];

    while (true) {
        ssize_t bytes = recv(socket_fd, buffer, sizeof(buffer), 0);

        if (bytes < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                // No data available right now, do other work
                std::cout << "No data available, doing other work...\n";
                usleep(100000);  // Sleep 100ms, then try again
                continue;
            }
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 5. Non-Blocking Socket with Error Handling (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            // Real error
            perror("recv error");
            break;
        }

        if (bytes == 0) {
            std::cout << "Connection closed\n";
            break;
        }

        // Process received data
        buffer[bytes] = '\0';
        std::cout << "Received: " << buffer << "\n";
    }
}
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 6. Socket Timeouts (Handling Slow Clients) (1/2)

Timeouts prevent your server from hanging indefinitely on slow or stalled clients. Critical for production servers to maintain responsiveness.

```
#include <sys/socket.h>
#include <sys/time.h>
#include <iostream>

bool set_recv_timeout(int socket_fd, int seconds) {
    struct timeval timeout;
    timeout.tv_sec = seconds;
    timeout.tv_usec = 0;

    if (setsockopt(socket_fd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) < 0) {
        perror("setsockopt SO_RCVTIMEO");
        return false;
    }
    return true;
}

bool set_send_timeout(int socket_fd, int seconds) {
    struct timeval timeout;
    timeout.tv_sec = seconds;
    timeout.tv_usec = 0;

    if (setsockopt(socket_fd, SOL_SOCKET, SO_SNDTIMEO, &timeout, sizeof(timeout)) < 0) {
        perror("setsockopt SO_SNDTIMEO");
        return false;
    }
    return true;
}

void handle_client_with_timeout(int client_fd) {
    // Set 5-second timeout for receives
    set_recv_timeout(client_fd, 5);

    // Set 10-second timeout for sends
    set_send_timeout(client_fd, 10);

    char buffer[1024];
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 6. Socket Timeouts (Handling Slow Clients) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    ssize_t bytes = recv(client_fd, buffer, sizeof(buffer), 0);

    if (bytes < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            std::cerr << "Client timeout - no data received in 5 seconds\n";
            close(client_fd);
            return;
        }
        perror("recv error");
        close(client_fd);
        return;
    }

    // Process data...
    buffer[bytes] = '\0';
    std::cout << "Received within timeout: " << buffer << "\n";
}
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 7. Getting Peer Information (1/2)

Useful for logging, access control, and debugging. In autonomous vehicles, you might want to verify connections come from authorized cloud servers.

```
#include <arpa/inet.h>
#include <sys/socket.h>
#include <iostream>
#include <cstring>

void print_peer_info(int socket_fd) {
    sockaddr_in peer_addr;
    socklen_t peer_len = sizeof(peer_addr);

    // Get peer address
    if (getpeername(socket_fd, (sockaddr*)&peer_addr, &peer_len) < 0) {
        perror("getpeername failed");
        return;
    }

    char ip_str[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &peer_addr.sin_addr, ip_str, INET_ADDRSTRLEN);

    std::cout << "Peer IP: " << ip_str << "\n";
    std::cout << "Peer Port: " << ntohs(peer_addr.sin_port) << "\n";
}

void print_local_info(int socket_fd) {
    sockaddr_in local_addr;
    socklen_t local_len = sizeof(local_addr);

    // Get local address
    if (getsockname(socket_fd, (sockaddr*)&local_addr, &local_len) < 0) {
        perror("getsockname failed");
        return;
    }

    char ip_str[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &local_addr.sin_addr, ip_str, INET_ADDRSTRLEN);

    std::cout << "Local IP: " << ip_str << "\n";
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 7. Getting Peer Information (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    std::cout << "Local Port: " << ntohs(local_addr.sin_port) << "\n";
}

// Usage in server
void handle_client(int client_fd) {
    std::cout << "=== Connection Info ===\n";
    print_local_info(client_fd);   // Server's local address
    print_peer_info(client_fd);    // Client's address
    std::cout << "=====================\n";

    // Handle client...
}
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 8. UDP Socket for Comparison (1/3)

TCP vs UDP Comparison:
- **TCP**: Connection-oriented, reliable, ordered delivery, use for critical data (HD maps, commands)
- **UDP**: Connectionless, unreliable, faster, use for sensor streams where occasional loss is acceptable (lidar point clouds)

```
#include <iostream>
#include <cstring>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <unistd.h>

// UDP Server (connectionless)
void udp_server_example() {
    // Create UDP socket (SOCK_DGRAM instead of SOCK_STREAM)
    int udp_fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (udp_fd < 0) {
        perror("socket failed");
        return;
    }

    sockaddr_in server_addr{};
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(6000);

    if (bind(udp_fd, (sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind failed");
        close(udp_fd);
        return;
    }

    std::cout << "UDP server listening on port 6000...\n";

    char buffer[1024];
    sockaddr_in client_addr;
    socklen_t client_len = sizeof(client_addr);

    while (true) {
        // No accept() needed - UDP is connectionless
        ssize_t bytes = recvfrom(udp_fd, buffer, sizeof(buffer) - 1, 0,
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 8. UDP Socket for Comparison (2/3)

```
// ...continued from previous slide
                                (sockaddr*)&client_addr, &client_len);

        if (bytes < 0) {
            perror("recvfrom failed");
            continue;
        }

        buffer[bytes] = '\0';

        char client_ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, INET_ADDRSTRLEN);

        std::cout << "Received from " << client_ip << ": " << buffer << "\n";

        // Echo back to sender
        sendto(udp_fd, buffer, bytes, 0,
               (sockaddr*)&client_addr, client_len);
    }

    close(udp_fd);
}

// UDP Client
void udp_client_example() {
    int udp_fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (udp_fd < 0) {
        perror("socket failed");
        return;
    }

    sockaddr_in server_addr{};
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(6000);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    const char* message = "Hello UDP";

    // No connect() needed for UDP (optional)
    ssize_t bytes = sendto(udp_fd, message, strlen(message), 0,
                           (sockaddr*)&server_addr, sizeof(server_addr));
// ...continues on next slide
```

TCP/IP Socket Fundamentals - Building Network Applications

Code Examples

📌 8. UDP Socket for Comparison (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    if (bytes < 0) {
        perror("sendto failed");
        close(udp_fd);
        return;
    }

    std::cout << "Sent: " << message << "\n";

    // Receive response
    char buffer[1024];
    socklen_t server_len = sizeof(server_addr);
    bytes = recvfrom(udp_fd, buffer, sizeof(buffer) - 1, 0,
                     (sockaddr*)&server_addr, &server_len);

    if (bytes >= 0) {
        buffer[bytes] = '\0';
        std::cout << "Received: " << buffer << "\n";
    }

    close(udp_fd);
}
```

What You Learned

Key Takeaways

- ✓ What is Network Programming?
- ✓ The Foundation
- ✓ Your Network Toolkit
- ✓ Server vs Client Architecture

→ Swipe to learn →